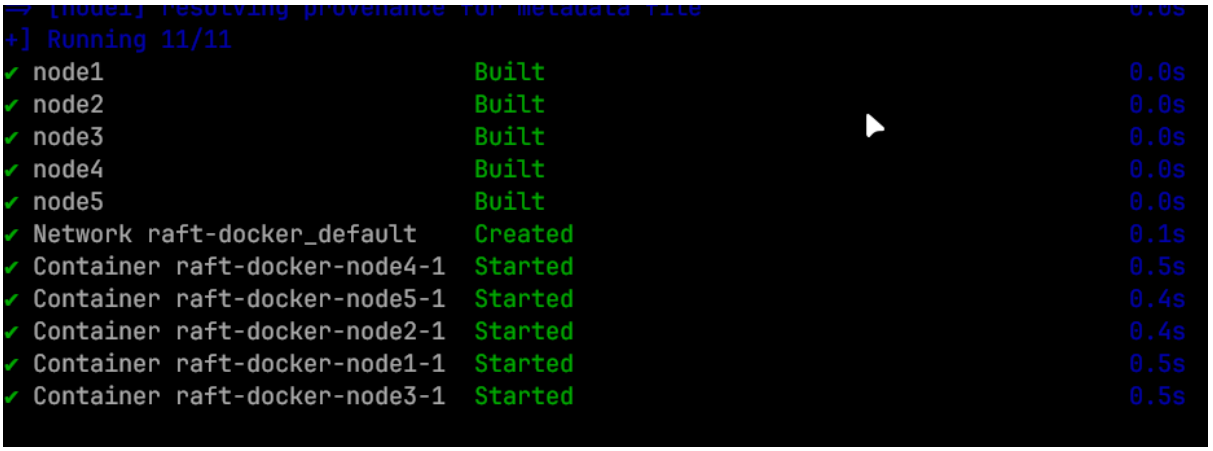
CB.SC.U4CYS23021
K.C.K.SANJAI
DISTRIBUTED SYSTEMS

RAFT

First ill paste the outputs followed by the codes so that it'll be easy to follow

Outputs:

Running the compose (contains multiple RAFT nodes running in different ports)



Running the demo.sh (that just looks at the node statuses and prints messages in such a way we can easily understand whats going on)



CODES:

raft.py

```
"""One Raft node, one container. Leader election, log replication,
commitment.
```

```
Everything is configured through the environment so the same image runs
any
cluster: NODE_ID, PEERS, PORT, HEARTBEAT_MS, ELECTION_MS,
RPC_TIMEOUT_MS.

Locking convention: self.lock guards every field below. Methods named
*_locked
assume the caller already holds it. An RPC is NEVER issued while
holding it --
the peer's handler takes its own lock and two nodes calling each other
would
deadlock instantly.
"""

import json
import os
import random
import threading
import time
import urllib.request
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

NODE_ID = os.environ["NODE_ID"]
PEERS = [p for p in os.environ["PEERS"].split(",") if p and p !=
NODE_ID]
PORT = int(os.environ.get("PORT", 8000))
HEARTBEAT = int(os.environ.get("HEARTBEAT_MS", 150)) / 1000
ELECTION = [int(x) / 1000 for x in os.environ.get("ELECTION_MS",
"800,1600").split(",")]
RPC_TIMEOUT = int(os.environ.get("RPC_TIMEOUT_MS", 300)) / 1000
MAJORITY = (len(PEERS) + 1) // 2 + 1

FOLLOWER, CANDIDATE, LEADER = "follower", "candidate", "leader"


def rpc(peer, path, body):
    """Fire one request. A dropped or timed-out reply is None; Raft just
retries."""
    req = urllib.request.Request(
        f"http://{peer}:{PORT}{path}",
        data=json.dumps(body).encode(),
        headers={"Content-Type": "application/json"},
```

```
    )
    try:
        with urllib.request.urlopen(req, timeout=RPC_TIMEOUT) as r:
            return json.loads(r.read())
    except Exception:
        return None


class Raft:
    def __init__(self):
        self.lock = threading.Lock()
        self.term = 0
        self.voted_for = None
        self.log = [{"term": 0, "cmd": ""}]  # sentinel: index 0 unused
        self.state = FOLLOWER
        self.leader = None
        self.commit = 0
        self.applied = 0
        self.votes = set()
        self.next = {}
        self.match = {}
        self.hb_deadline = 0
        self.reset_timer_locked()

    # --- helpers (lock held) -----------------------------------------------
    def last_locked(self):
        return len(self.log) - 1

    def reset_timer_locked(self):
        self.deadline = time.monotonic() + random.uniform(*ELECTION)

    def step_down_locked(self, term):
        """Any message carrying a higher term drags a node back to follower."""
        self.state, self.term, self.voted_for = FOLLOWER, term, None
        self.reset_timer_locked()

    def start_election_locked(self):
        self.state, self.term, self.voted_for = CANDIDATE, self.term + 1, NODE_ID
        self.votes = {NODE_ID}
        self.leader = None
```

```
        self.reset_timer_locked()
        print(f"[{NODE_ID}] timeout, campaigning for term {self.term}")

    def become_leader_locked(self):
        self.state, self.leader, self.hb_deadline = LEADER, NODE_ID, 0
        # Assume every follower matches us, then walk back on rejection.
        self.next = {p: self.last_locked() + 1 for p in PEERS}
        self.match = {p: 0 for p in PEERS}
        self.match[NODE_ID] = self.last_locked()
        print(f"[{NODE_ID}] WON term {self.term} with {len(self.votes)}
votes")

    def advance_commit_locked(self):
        for n in range(self.last_locked(), self.commit, -1):
            if self.log[n]["term"] != self.term:
                break  # 5.4.2: a leader may only commit entries from
its own term
            if sum(v >= n for v in self.match.values()) >= MAJORITY:
                self.commit = n
                return

    # --- background loops
-------------------------------------------------
    def ticker(self):
        """Heartbeat when leader, otherwise count down to an
election."""
        while True:
            time.sleep(0.02)
            with self.lock:
                now = time.monotonic()
                if self.state == LEADER and now >= self.hb_deadline:
                    self.hb_deadline = now + HEARTBEAT
                elif self.state != LEADER and now >= self.deadline:
                    self.start_election_locked()
                else:
                    continue
                job = self.send_append if self.state == LEADER else
self.send_vote
            for p in PEERS:  # outside the lock: these block on the
network
                threading.Thread(target=job, args=(p,),
daemon=True).start()
```

```
    def applier(self):
        while True:
            time.sleep(0.02)
            with self.lock:
                entries = self.log[self.applied + 1 : self.commit + 1]
                self.applied = self.commit
            for e in entries:
                print(f"[{NODE_ID}] applied {e['cmd']!r}")

    # --- outbound RPCs
-----------------------------------------------------
    def send_vote(self, peer):
        with self.lock:
            if self.state != CANDIDATE:
                return
            term = self.term
            body = {"term": term, "from": NODE_ID, "last_index":
self.last_locked(),
                    "last_term": self.log[-1]["term"]}
        reply = rpc(peer, "/vote", body)
        if not reply:
            return
        with self.lock:
            if reply["term"] > self.term:
                return self.step_down_locked(reply["term"])
            # We may have been deposed and re-elected while this was in
flight.
            if self.state != CANDIDATE or self.term != term or not
reply["granted"]:
                return
            self.votes.add(peer)
            if len(self.votes) >= MAJORITY:
                self.become_leader_locked()

    def send_append(self, peer):
        with self.lock:
            if self.state != LEADER:
                return
            term, prev = self.term, self.next[peer] - 1
            body = {"term": term, "from": NODE_ID, "prev_index": prev,
                    "prev_term": self.log[prev]["term"],
                    "entries": self.log[prev + 1 :], "commit":
self.commit}
```

```
        reply = rpc(peer, "/append", body)
        if not reply:
            return
        with self.lock:
            if reply["term"] > self.term:
                return self.step_down_locked(reply["term"])
            if self.state != LEADER or self.term != term:
                return
            if reply["success"]:
                self.match[peer], self.next[peer] = reply["match"],
reply["match"] + 1
                self.advance_commit_locked()
            elif self.next[peer] > 1:
                self.next[peer] -= 1  # back up one and retry next
heartbeat

    # --- inbound RPCs (called on HTTP server threads)
-----------------------
    def on_vote(self, m):
        with self.lock:
            if m["term"] > self.term:
                self.step_down_locked(m["term"])
            # Up-to-date (5.4.1) compares term first, then length.
            granted = (
                m["term"] == self.term
                and self.voted_for in (None, m["from"])
                and (m["last_term"], m["last_index"])
                >= (self.log[-1]["term"], self.last_locked())
            )
            if granted:
                self.voted_for = m["from"]
                self.reset_timer_locked()
            return {"term": self.term, "granted": granted}

    def on_append(self, m):
        with self.lock:
            if m["term"] > self.term:
                self.step_down_locked(m["term"])
            if m["term"] < self.term:
                return {"term": self.term, "success": False, "match": 0}

            self.state, self.leader = FOLLOWER, m["from"]
            self.reset_timer_locked()
```

```
            # Consistency check: our log must match the leader's at
prev_index.
            prev = m["prev_index"]
            if prev > self.last_locked() or self.log[prev]["term"] !=
m["prev_term"]:
                return {"term": self.term, "success": False, "match": 0}

            for i, e in enumerate(m["entries"], start=prev + 1):
                if i <= self.last_locked():
                    if self.log[i]["term"] == e["term"]:
                        continue  # already have it
                    del self.log[i:]  # conflict: drop it and everything
after
                self.log.append(e)

            if m["commit"] > self.commit:
                self.commit = min(m["commit"], self.last_locked())
            return {"term": self.term, "success": True, "match":
self.last_locked()}

    def on_submit(self, m):
        with self.lock:
            if self.state != LEADER:
                return {"ok": False, "leader": self.leader}
            self.log.append({"term": self.term, "cmd": m["cmd"]})
            self.match[NODE_ID] = self.last_locked()
            return {"ok": True, "index": self.last_locked()}

    def status(self):
        with self.lock:
            return {"id": NODE_ID, "state": self.state, "term":
self.term,
                    "commit": self.commit, "leader": self.leader,
                    "log": [e["cmd"] for e in self.log[1:]]}


class Handler(BaseHTTPRequestHandler):
    ROUTES = {"/vote": "on_vote", "/append": "on_append", "/submit":
"on_submit"}

    def do_POST(self):
```

```
        body =
json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        self.reply(getattr(raft, self.ROUTES[self.path])(body))

    def do_GET(self):
        self.reply(raft.status())

    def reply(self, obj):
        data = json.dumps(obj).encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)

    def log_message(self, *args):
        pass  # silence the per-request access log


if __name__ == "__main__":
    raft = Raft()
    threading.Thread(target=raft.ticker, daemon=True).start()
    threading.Thread(target=raft.applier, daemon=True).start()
    print(f"[{NODE_ID}] up on :{PORT}, peers {PEERS}, majority
{MAJORITY}")
    ThreadingHTTPServer(("", PORT), Handler).serve_forever()
```

Dockerfile

```
FROM python:3.12-alpine
WORKDIR /app
COPY raft.py .
CMD ["python", "-u", "raft.py"]
```

Dockercompose

```
# Five Raft nodes. To resize the cluster, add/remove a service block
and
# update PEERS. Source is mounted, so editing raft.py needs only a
restart.
x-env: &env
 PEERS: node1,node2,node3,node4,node5
```

```
 PORT: "8000"
 HEARTBEAT_MS: "150"
 ELECTION_MS: "800,1600"
 RPC_TIMEOUT_MS: "300"

x-node: &node
 build: .
 image: raft-py
 volumes:
  - ./raft.py:/app/raft.py:ro
 stop_grace_period: 1s

services:
 node1:
   <<: *node
   environment: {<<: *env, NODE_ID: node1}
   ports: ["8001:8000"]
 node2:
   <<: *node
   environment: {<<: *env, NODE_ID: node2}
   ports: ["8002:8000"]
 node3:
   <<: *node
   environment: {<<: *env, NODE_ID: node3}
   ports: ["8003:8000"]
 node4:
   <<: *node
   environment: {<<: *env, NODE_ID: node4}
   ports: ["8004:8000"]
 node5:
   <<: *node
   environment: {<<: *env, NODE_ID: node5}
   ports: ["8005:8000"]
```

demo.sh

```
#!/usr/bin/env bash
# Elect, replicate, kill the leader, elect again, restart and
back-fill.
# Needs curl only. Node N is published on port 800N.
set -e
cd "$(dirname "$0")"
```

```
PORTS="8001 8002 8003 8004 8005"

leader_port() {
 for p in $PORTS; do
   curl -sm1 localhost:$p/status 2>/dev/null | grep -q '"state":
"leader"' && { echo "$p"; return; }
 done
}

show() { for p in $PORTS; do curl -sm1 localhost:$p/status || echo
"(port $p down)"; echo; done; }

submit() { curl -s -X POST "localhost:$1/submit" -d "{\"cmd\":\"$2\"}";
echo; }

# --force-recreate so a cluster left running from a previous poke
starts empty.
docker compose up -d --build --force-recreate
sleep 4

echo " --- final state ---"
show
echo "clean up with: docker compose down"
```